You are currently viewing the Trifork Blog, to view our full website please go to Trifork.com

Integrating Flyway In A Spring Framework Application

December 9, 2014 | Written by **Dadepo Aderemi**

7 Comments

This post is about how to integrate Flyway into a Spring/JPA application for database schema migration. To skip all the preambles and get straight to the instructions, jump to Project's Dependencies Set-up

Flyway is a database migration tool which helps do to databases, what tools like git/svn/mercurial does for source code...which is versioning. With Flyway you can easily version your database: create, migrate and ascertain its state, structure, together with its contents. It basically allows you to take control of your database, and be able to recreate it across different environment or different versions of the application it runs with, while keeping track of the chronological changes made.

I recently worked on a project where such database schema migration was needed. And we found Flyway a good tool for the job. Prior to using Flyway, all of the schema updates that had to be done were delivered to the client in separate SQL files, with instruction on how they should be run.

The short coming of such a process soon became apparent. It was burdensome and error prone.

First of all, it calls for extra operation activities on the part of the client as they have to run whatever update scripts that are provided and ensure the database schema is in the required state before application deployment – Not an ideal process. An application should be as self-contained as possible, so when delivered, it should be deployable without much hassle on the client's part.

Also it unnecessarily exposes the internals of the database state. Having the database modified outside of the application does not guarantee its integrity, especially when the database modification is being done by a party who is not privy to the internals of the application the database is running with.

So when the time came to upgrade the version of the application in production to a newer version, we decided to re-evaluate the process through which we provide schema upgrades. This is when Flyway entered the picture.

As mentioned, Flyway is a database migration tool. It can be used via its command line tool, via a maven plug-in or programmatic-ally from within an application, but Since the application I was working on was built with Spring framework, and uses a JPA backed database (provided by Hibernate), I then set about on how to have Flyway integrated into such an application. This post captures the typical step that would be involved.

Recently posted on Trifork Blog

5 Proven AI use cases for manufacturers

Modern Manufacturing

Functional Friction

Innovation across Silos

Parallel Short-Circuiting With Reactive Programming

AI-powered empathy for customer contact

Verifying Coding Standards Using Static Code Analysis Techniques

Trifork highlighted vacancies:

Trifork in Amsterdam is looking for developers: Medior Java developer

Other International Trifork vacancies

Blog categories

Agile Android AngularJS Apache Mahout Artificial Intelligence/ Machine Learning Axon Axon Framework Business C# Cloud Native Conference Content Management DevOps Digital Transformation Docker Education Enterprise Search

But before getting into the steps, a little overview of how Flyway works would help have a clear understanding of the integration instructions.

How Flyway works. An Overview

Schema changes and content modification (adding, modifying or deleting contents in database tables) are done via SQL statements. What Flyway does is to provide a mechanism in which these changes can be recorded and versioned. This it does by 1) Being responsible for applying the schema changes specified in the SQL scripts and 2) By keeping an internal meta-data table named SCHEMA_VERSION through which it keeps track of various information regarding the applied scripts: when it was applied, by whom it was applied, description of the migration applied, its version number etc.

Flyway allows for the SQL scripts (which are frequently referred to as migrations) to be provided in the form of plain old SQL script files or via Java code. The important thing though, is that, whichever form the migration is written, the files need to be named following a convention, which is explained below.

For more information on how Flyway works, check the How Does Flyway Work in the project website.

Now with the brief overview out of the way, we can go ahead and look at the instructions.

Integrating Flyway into a JPA, Spring backed application.

When integrating Flyway into a JPA, Spring application, what you want to do is to have Flyway kick in, find the migration files (either in SQL or JAVA) and apply the necessary modifications to the database at application start-up. The following steps show how to achieve this.

1. Project's Dependencies Set-up

The first thing to do is to add the needed project dependencies, so apart from the Spring/JPA dependencies, include Flyway as a dependency. If you are using Maven, then the following lines should be added to the dependencies section of your POM.xml:

```
<dependency>
    <groupId>org.flywaydb</groupId>
    <artifactId>flyway-core</artifactId>
    <version>${Flyway.version}</version>
</dependency>
```

In case you are not using maven for your dependency management, the process of including Flyway should be trivial in whatever tool you are using.

2. Configuring Flyway to integrate into Spring's Container

The next steps would be the configuration needed to have Spring pick up Flyway as a managed bean and have it play nicely with other beans it would need to work it...to be specific the entity manager factory.

If you are using XML to configure spring the necessary configuration would look thus:

```
<!-- Flyway configuration -->
<bean id="flyway" class="org.Flyway.core.Flyway" init-method="migrate">
    <property name="baselineOnMigrate" value="true" />
    <property name="locations" value="filesystem:/path/to/migrations/" />
    <property name="dataSource" ref="dataSource" />
</bean>
```

From The Trenches Frontend General Groovy GWT Hippo HTML5 Internet of Things iPad iPhone Java Knowledge Liferay Magnolia Microservices Mobile Newsletter NoSQL NoSQL & Search Objective C Recruitment Software Development Spring Swift System Administration Training User Experience Windows Phone

GOTO Academy

Upcoming IT courses in Amsterdam, NL:

Designing Microservices with Sam Newman | October 09-11, 2019

Core Spring with Thomas Zeeman | Nov 12 - 15, 2019

Modern JVM Development with Garth Gilmour | Nov 27-29, 2019

Designing & Building Great APIs with Mike Amundsen | Dec 4-6, 2019

GOTO Conferences

- GOTO Berlin
Oct 21 - Oct 25, 2019

- GOTO Copenhagen
Nov 18 - 22, 2019

- GOTO Chicago
Apr 27- May 1, 2020

```
<!-- Entity Manager Factory configuration -->
<bean id="entityManagerFactory" class="o.s.orm.jpa.LocalContainerEntityManagerFactoryBe depends-on="flyway">
    <property name="dataSource" ref="dataSource" />
    <property name="jpaVendorAdapter">
        <bean class="o.s.orm.jpa.vendor.HibernateJpaVendorAdapter">
            <property name="database" value="${jpa.database}"/>
        </bean>
    </property>
</bean>

<!-- an example of a data source -->
 <bean id="dataSource" class="o.a.commons.dbcp.BasicDataSource" lazy-init="true" destroy-method="close">
        <property name="driverClassName" value="org.hsqldb.jdbcDriver"/>
        <property name="url" value="jdbc:hsqldb:mem:qade"/>
        <property name="username" value="sa"/>
        <property name="password" value=""/>
  </bean>
```

- GOTO Amsterdam
June 8-11, 2020

If you are using JavaConfig over XML then the configuration would look thus:

```
@Configuration

public class AppConfig {

@Bean(initMethod = "migrate")
Flyway flyway() {
Flyway flyway = new Flyway();
flyway.setBaselineOnMigrate(true);
flyway.setLocations("filesystem:/path/to/migrations/");
flyway.setDataSource(dataSource());
return flyway;
}

@Bean @DependsOn("flyway")
EntityManagerFactory entityManagerFactory() {
LocalContainerEntityManagerFactoryBean bean = new LocalContainerEntityManagerFactoryBean();
bean.setDataSource(dataSource());
// other configurations
return bean.getObject();
}

@Bean
DataSource dataSource() {
DataSource dataSource = new BasicDataSource();
// data source configuration
return dataSource;
}
}
```

Some additional things to take note of in the configurations:

Flyway Configuration.
When configuring Flyway you need to specify the *init method* (which was done using the *init-method* property in xml and *initMethod* property of the *@Bean* annotation in JavaConfig)

This is used to instruct Spring to call the *migrate()* method once all the properties of the Flyway bean has been initialized. The *migrate()* method is what is responsible for performing the migration logic: that is finding the migration scripts, applying them and keeping a tab of successful migrations etc. Which is why it is specified as the init method as you would want this to be executed as soon as the Flyway bean is initialized in Spring's container.

The *baseLineOnMigrate* is also another interesting part of the configuration. It comes in handy when Flyway is initially used for the first time and no SCHEMA_VERSION table exists. It instructs Flyway that before the migration scripts are applied, it should create a migration entry within the

SCHEMA_VERSION table which would serve as the baseline, and thus the available migration script would only be applied if their version is higher than the baseline version.

The *dataSource* is used to specify the dataSource. Nothing interesting about that.

The last item of interest in the configuration is the *location* through which you specify where Flyway should find the migration scripts. Flyway has the ability to scan either the filesystem or classpath for these migration scripts. In the above example the value given to the location is prefixed with "*filesystem:*" indicating to Flyway to use its file system scanner to locate the migration scripts. If no prefix is speicified or "*classpath:*" is used as the prefix, then Flyway uses its classloader scanner to locate the migration scripts.

Entity Manager Factory Configuration.

The one thing to take note in the Entity Manager Factory's configuration is the *depends on* property that is set to reference the Flyway bean. What this does is to ensure that the Flyway bean is always created before the Entity Manager Factory bean. Which is what you would want as you need Flyway to already have been created, do its thing before the Entity Manager Factory kicks in. In the JavaConfig this is specified using the *@DependsOn* annotation.

3. Writing Migration Scripts

Now that you have the necessary beans configured, the next thing to do is to create the migration scripts which contains the necessary schema updates that needs to be applied to the database during a migration, As specified earlier, Flyway supports migration scripts written in plain old SQL or in Java.

Whichever method you use, all you have to do is to have the migration file named appropriately, following the convention required by Flyway and have them in the location specified in the configuration.

Why a naming convention? The naming convention is required, as this is the default way Flyway keeps track of its versioning (I say default because when using Java, there is the possibility to override this naming mechanism. This is explained below). It uses it to determine the order in which migration scripts needs to be applied and to keep track of the scripts applied and ones pending.

The naming convention is as follows:

V<VERSION>__<DESCRIPTION>.<FORMAT>

Where **<VERSION>** is a numeric value that can contain a point (".") or an underscore ("_") for example 3, 3.1, 3_1 are all valid for the version. (Not that if an underscore is used, it would be turned into a dot "." at runtime by Flyway)

The double underscore, __ is what is used to separate the version number from the description.

<DESCRIPTION> is a short description of what the scripts contained in the file is about. It would be extracted and added to the description column of the schema_version table.

For the **<FORMAT>,** you can have either sql or java depending on the method being used to supply the migrations.

So the following are all valid names

V3__description_of_migration.sql
V3.1__description_of_migration .java
V3_1__description_of_migration .sql

If the migrations are specified in SQL there is little else that can be done apart from specifying the schema updates and naming them appropriately. On the other hand, with Java we have a little more flexibility. I would quickly take a look at how to use Java to write the migration and some of its available features not present when using SQL.

3. Writing Flyway Migration in Java.

As with migration written in SQL you need to have the Java migration in the location specified in the configuration. You then also need to have the class in which the sql migration are written implement the *JdbcMigration* or SpringJdbcMigration interface.

The difference between these two interfaces is that *jdbcMigration* makes a Connection object available for interacting with the database while *SpringJdbcMigration* makes a JdbcTemplate object available for database interaction.

So a migration script written in Java may look like this:

```
1  import org.flywaydb.core.api.migration.spring.SpringJdbcMigrat
2  import org.springframework.jdbc.core.JdbcTemplate;
3
4  public class V2__moveDBStateToV2 implements SpringJdbcMigration {
5
6  @Override
7  public void migrate(JdbcTemplate jdbcTemplate) throws Exception {
8  // programmatically change the database state using JdbCTemplate
9  }
10
11 }
```

And that is it. This way you can perform complex checks and operations needed to move the database state into a required state.

Apart from that, the Java option also provides some additional functionality like allowing you to disregard the naming convention and instead have the version and description specified in the Java class. This is done by implementing the MigrationInfoProvider Interface. For example:

```
1  public class WhateverName implements SpringJdbcMigration, MigrationInfoProvider {
2
3  @Override
4  public void migrate(JdbcTemplate jdbcTemplate) throws Exception {
5  // programmatically change the database state using JdbCTemplate
6  }
7
8  @Override
9  public MigrationVersion getVersion() {
10 return MigrationVersion.fromVersion("2"); //return 2 as version
11
12 }
13
14 @Override
15 public String getDescription() {
16 return "Moves DB to State Two";
17 }
18
19 }
```

Some closing thoughts.

When I first approached Flyway as the tool to be used for database migration, one of the first thoughts that came to mind was, why use an tool when I already have hibernate hbm2ddl which can also be used to modify the

database schema. I already have hibernate being the JPA provider, why an external tool?

The answer is simple. Hibernates hbm2ddl is in no way as flexible or as powerful compared to a database migration tool like Flyway. At its core, hbm2ddl could be seen as just a tool for extending your database to keep it in line with new additions to the backing entity. It does not have the ability to modify field names, transfer data from one column to another etc. Tasks which always pop up in any real life database migration situation.

So if you need to do proper database migration and versioning, a tool like Flyway is quite handy.

A thing to also note is that Flyway can be used as a standalone migration tool. This is possible via Flyway's robust command line tool, whose instruction can be found here. Flyway also provides callbacks which can be used as extension points to hook into Flyway's lifecycle to perform advance operations. The instruction on the callback can be found here.

All in all, Flyway really shines in that it gets the job done and it gets it done well. It is relatively simple with little learning curve and flexible. The documentation and JavaDoc is also decent, which makes working with it painless.

From The Trenches, Java, Software Development, Spring Flyway, Java, Spring

ANWB Big data Proof of Concept Pen and Paper Vs. Sketching Tools

6 Comments

1 Ping/Trackback

Frans Flippo

December 10, 2014 - 15:26

Great comprehensive post on Flyway! We used Flyway in the Financieel Dagblad project from the start and it's doubtlessly saved us a lot of time and manual error.

When your team is working on different features in parallel in different branches, it can happen that version numbers clash. What we've done is agreed on a convention for "reserving" a version number by placing a placeholder file on the master branch. In addition, we use outOfOrder mode in Flyway (flyway.setOutOfOrder) so that no matter in what order the branches are eventually merged into the master, the migrations will be run. This requires, of course, that there are no conflicts in your database migrations between the two branches.

Dadepo Aderemi Author

December 11, 2014 - 09:46

Yes indeed, there is the need to manage versioning to prevent clashes in the scenario you describe...Another trick I have come across in the wild to deal with this is having a work flows that revolves around having the version number generated as timestamps!

Denis Tunovic

January 27, 2015 - 10:51

Any thoughts on pros/cons in comparison with Liquibase?

Roman Schaller

June 15, 2015 - 16:56

Thank you for this comprehensive post. It was very helpful to me. Although I had to rename the method entityManagerFactory to entityManagerFactoryBean to make sure that flyway starts before jpa.

Migrating Hibernate Data | coding Pingback
August 14, 2015 - 05:15

[...] From this Spring-Flyway guide: [...]

Keith Roberts

October 3, 2015 - 00:16

This is a very nice article. Thanks for posting it. I've done quite a bit of googling for Flyway articles and am not able to find a good description for how to migrate actual data, different data requirements for different environments. Looks like writing scripts in an env specific location would work best for my use case. Too much env specific SQL to use placeholders. What I can't figure out is how to specify which location to use for a specific env in the Spring boot application.

Zhandos

November 6, 2015 - 15:40

Thanks for the article!
I have never heard about db migrations in java before. But last months I was programming in C# (.NET), and there I saw great tool to manage db schema state. So I googled for Java and found Flyway.

But, in .NET migration tool is better than Flyway (or may be I havent used it yet, just reading tutorials), because in .NET you dont write migration code, migration instrument writes code for you! you just type "add migration mig_name", and you have the code generated for your last entity modifications. Then you run "update database" and script is done.

Is there such tool for JAVA?

Comments are closed

Trifork Blog © 2019 Trifork A/S. All rights reserved Trifork